

基于机器学习的组合优化技术调研报告

何子腾 王睿琛 周韧平 赵子钰

April 8, 2025

Abstract

1 问题背景

2 问题一：生产可行解 (Primal task)

2.1 代表性论文与方法综述

2.1.1 Reinforcement Learning for Solving the Vehicle Routing Problem (NeurIPS 2018)

作者： Mohammadreza Nazari, Afshin Oroojlooy, Lawrence V. Snyder, Martin Takáč

方法详解： 该论文 [NOST18]提出了一种强化学习框架解决车辆路径问题（VRP），其核心思路如下：

- **模型架构：** 采用基于注意力机制的指针网络（Pointer Network），使用编码器-解码器结构对客户节点进行建模。编码器处理客户位置信息，解码器以自回归方式逐步生成访问序列。
- **训练方式：** 使用策略梯度方法（REINFORCE）进行端到端训练，以路径总长度的负值为奖励信号，通过与贪婪策略作为baseline 结合，减少梯度方差，提升训练稳定性。
- **约束建模：** 模型通过掩码机制处理容量约束与访问合法性，确保每一步选择的节点满足可行性条件。
- **推理阶段：** 训练完成后，可直接应用于任意同规模的VRP实例，无需重训练，具备一定泛化能力。

创新点总结：

1. 采用端到端强化学习策略，避免手工设计启发式；
2. 指针网络结构结合问题特性，天然适配序列生成任务；
3. 强化学习方式直接在解空间中生成高质量可行解，避免传统方法中的复杂搜索过程。

2.1.2 Combinatorial Optimization with Graph Convolutional Networks and Guided Tree Search (NeurIPS 2018)

作者: Zhuwen Li, Qifeng Chen, Vladlen Koltun

方法详解: 该方法 [LCK18]旨在通过学习型搜索策略, 主动生成结构良好且可行的解。该论文提出一种结合图卷积神经网络与引导树搜索的框架, 用于求解图结构组合优化问题, 方法流程如下:

- **问题建模:** 将优化问题转化为图结构, 节点代表决策变量或图元素, 边代表其间约束关系。
- **GCN 表征学习:** 使用多层图卷积网络学习节点表示, 编码局部与全局结构信息。
- **引导搜索策略:** 基于节点嵌入输出每个节点在当前解构造中的得分, 引导搜索过程逐步构建高质量可行解, 避免暴力枚举。
- **多样化解生成:** 引入多次采样和集成方法, 提升解集的覆盖率与多样性。

创新点总结:

1. 首次结合GCN与启发式搜索, 实现结构化评分函数学习;
2. 兼顾表示能力与搜索效率, 适应多种组合优化问题 (如TSP、MaxCut);
3. 可扩展至大规模实例, 具备良好泛化能力与可转移性。

2.1.3 Exact Combinatorial Optimization with Graph Convolutional Neural Networks (NeurIPS 2019)

作者: Maxime Gasse, Didier Ch é telat, Nicola Ferroni, Laurent Charlin, Andrea Lodi

方法详解: 该论文 [GCF⁺19]研究如何将学习方法嵌入精确组合优化框架 (如分支定界), 提高MILP 求解器效率。尽管该方法主要用于引导精确求解器的分支策略, 但其学习到的启发式得分函数可用于辅助初始可行解的快速构造。方法关键如下:

- **图结构建模:** 将MILP 问题建模为变量-约束二部图, 变量和约束为节点, 边表示依赖关系。
- **图神经网络编码:** 使用专门设计的bipartite GCN 对图结构进行编码, 提取变量嵌入向量。
- **强分支模仿学习:** 以专家级启发式 (如Strong Branching) 生成的选择顺序作为监督信号, 训练模型输出每个变量的分支优先级得分。
- **与分支定界集成:** 将模型输出用于排序候选变量, 在不牺牲最优性的前提下, 加快分支定界过程。

创新点总结:

1. 提出将图神经网络集成进精确求解器的方法, 实现机器学习与传统优化结合
2. 通过图神经网络学习变量重要性评分函数, 引导优化过程向可行解快速收敛, 加速解的生成
3. 支持跨问题迁移, 首次在大规模问题上超越手工设计启发式规则。

2.2 相关代码介绍

2.2.1 Reinforcement Learning for Solving the Vehicle Routing Problem (NeurIPS 2018)

代码仓库: <https://github.com/OptMLGroup/VRP-RL>

OptMLGroup/VRP-RL 的代码结构围绕强化学习解决路径规划问题设计, 核心模块包括环境、模型、训练流程与数据处理。代码结构如下:

- train.py: 主训练文件
- envs: 存放TSP/VRP 环境实现, 其中定义了TSP (旅行商问题) 和VRP (车辆路径问题) 的强化学习环境, 负责状态管理、动作空间、奖励计算等核心逻辑。关键参数input_dim 默认为2 (对应二维坐标, 如经纬度)。在真实应用中用户需要将其调整至更高的维度以适应复杂距离计算 (如道路距离而非直线距离)
- models: 定义强化学习策略网络, 网络结构同 2.1.1。
- utils: 辅助函数 (如数据加载、日志记录)

2.2.2 Combinatorial Optimization with Graph Convolutional Networks and Guided Tree Search

代码仓库: <https://github.com/isl-org/NPHard> 旨在利用图卷积网络 (GCN) 和引导树搜索 (Guided Tree Search) 来解决最大独立集问题。代码结构如下:

- data/: 包含用于训练和测试的图数据集。
- gcn/: 实现了图卷积网络的相关代码。
- kernel/: 包含底层的算法实现或优化代码
- model/: 定义了用于求解最大独立集问题的模型结构。
- demo.py 和demo_parallel.py: 提供了示例脚本, 展示如何使用训练好的模型进行推理, 可能包括并行处理的示例。
- train.py: 用于训练图卷积网络模型的脚本。

值得注意的是, 目前这个代码仓库已经不再维护, 在具体使用时可能存在安全风险。

2.2.3 Exact Combinatorial Optimization with Graph Convolutional Neural Networks

代码仓库: <https://github.com/ds4dm/learn2branch>. 该项目利用Ecole 库、PyTorch 和PyTorch Geometric 来简化原始实现, 并旨在通过学习分支定界 (Branch-and-Bound) 算法中的变量选择策略, 提升求解混合整数线性规划 (MILP) 问题的效率。代码结构如下:

- model/: 包含GNN 模型的定义和相关代码。
- 02_generate_dataset.py: 用于生成监督学习数据集的脚本。
- 03_train_gnn.py: 负责训练GNN 模型的脚本。
- 04_evaluate.py: 用于评估训练好的模型性能的脚本。
- utilities.py: 提供辅助函数和工具模块。

2.3 竞赛：Machine Learning for Combinatorial Optimization (NeurIPS 2021)

该比赛 [GCC+22]旨在探索机器学习（ML）如何改进组合优化（CO）问题的求解，尤其关注将ML与传统CO方法（如MIP求解器、启发式算法）结合，以提升求解效率或解的质量。

2.3.1 问题背景

传统CO方法（如分支定界、动态规划）在复杂或大规模问题上可能计算成本高昂，而ML可以学习问题结构、加速搜索或直接生成可行解。比赛目标是设计ML+CO的混合方法，解决三类经典NP难问题：

- 装箱问题（Bin Packing）：将物品放入最少数量的固定容量箱子。
- 作业车间调度（Job Shop Scheduling）：优化任务在机器上的执行顺序以最小化完成时间。
- 车辆路径问题（VRP）：规划车辆路线以最小化总距离，满足客户需求。

2.3.2 数据集

数据由比赛组织者通过公开基准实例或随机生成，确保多样性和复杂性。对装箱问题来说，物品尺寸和箱子容量服从特定分布（如均匀分布、偏态分布）。作业车间调度问题中，任务数量和机器数量可变，处理时间随机生成。车辆路径问题中，客户位置、需求量和车辆容量随机生成。

2.3.3 评估标准

每个问题均采用领域特定指标，重点关注：解的质量（与最优解的差距）、计算效率和泛化能力，具体来说，装箱问题中测量所用箱子数量的减少，作业车间调度中测量最大完工时间的缩短，VPR问题中测量总行驶距离的降低。

2.3.4 优胜方案及关键方法

比赛队伍主要从图神经网络、强化学习、Transformer等技术出发提出了相应解决方案，也有部分工作和传统求解器相结合（如Google的《Learning to Branch》）。

华为EI-OROAS团队适用图卷积神经网络（GCNN）的技术路线 [BDZ21]。GCNN通过模仿学习来近似强分支（Strong Branching, SB）规则，将分支策略建模为GCNN，利用MILP问题的二分图表示。该方法将分支变量选择建模为分类问题，训练分类模型以预测SB代理的分支变量选择，使用softmax交叉熵作为训练损失函数。文章指出，神经网络的大小与推理速度密切相关。较大的模型虽然分类准确率更高，但推理速度较慢，可能会影响最终的奖励值。例如，在物品放置数据集上，较小的模型（8维嵌入）表现出更快的推理速度和更好的奖励值。

3 问题二：提供分支策略（Dual task）

3.1 代表性论文与方法综述

3.1.1 Learning to Branch in Mixed Integer Programming(AAAI 2016)

作者：Elias B. Khalil, Pierre Le Bodic, Le Song, George Nemhauser, Bistra Dilkina

方法详解：该论文[KLBS+16]提出首个基于机器学习的混合整数规划（MIP）动态分支框架，通过模仿强分支策略实现高效变量选择。方法流程如下：

- **三阶段框架设计：**

- 数据收集阶段：在前 θ 个节点（默认500）运行强分支（SB），记录候选变量的72维特征向量（静态/动态特征）及二元标签（基于SB得分阈值 $\alpha=0.2$ ）
- 模型训练阶段：使用SVM-rank算法学习特征权重向量，通过成对排序损失函数保持与SB决策的一致性
- 预测分支阶段：基于学习模型对候选变量评分，选择最高分变量分支

- **特征工程：** 包含18个静态特征（目标系数、约束参与度统计等）和54个动态特征（伪成本、活跃约束系数统计等），采用二次多项式扩展生成交互特征

- **在线学习机制：** 每个MIP实例独立训练模型，避免离线训练依赖

创新点总结：

1. 首次将学习排序（learning-to-rank）引入MIP分支策略，建立特征空间与分支质量的映射关系
2. 提出实例自适应的在线学习范式，突破传统启发式策略的输入无限制
3. 设计混合特征体系，平衡问题结构特征（如约束参与度）与搜索动态特征（如伪成本历史）
4. 实验验证新方法较伪成本分支（PC）减少36%节点，较混合分支（SB+PC）减少16%节点，接近商业求解器CPLEX默认策略水平

3.1.2 Hybrid Models for Learning to Branch (NeurIPS 2020)

作者： Prateek Gupta, Maxime Gasse, Elias B. Khalil, M. Pawan Kumar, Andrea Lodi, Yoshua Bengio

方法详解： 该论文[GGK+20]提出一种混合架构与训练协议，解决传统GNN模型在CPU环境下的推理效率瓶颈，同时保持分支决策质量，核心流程如下：

- **混合架构设计：**

- 根节点采用GNN提取全局图结构特征（变量-约束二分图），生成高维嵌入。
- 后续树节点仅提取轻量级分支候选特征（如伪成本、分数变量值），通过MLP快速决策。
- 设计多种特征融合方案（如FiLM调制、HyperSVM参数预测），平衡表达力与计算开销。

- **训练协议优化：**

- **端到端训练：** 联合优化GNN与MLP参数，对齐根节点与树节点表示。
- **知识蒸馏：** 用预训练GNN输出指导MLP训练，提升泛化能力。
- **辅助任务：** 通过变量嵌入分离度损失（ED/MHE）增强特征多样性。
- **深度加权损失：** 根据节点深度调整损失权重（如Sigmoidal加权），强化根节点决策重要性。

创新点总结：

1. 首次提出CPU友好的GNN-MLP混合架构，在根节点捕捉全局结构，树节点轻量推理，解决GNN的GPU依赖问题。
2. 融合多种训练策略（知识蒸馏、辅助任务、深度加权），有效缓解分布偏移与过拟合，提升模型泛化性。
3. 实验验证在设施选址、组合拍卖等4类MILP问题上，CPU推理时间较SCIP默认策略减少26%，节点数接近强分支（SB）水平。
4. 支持跨规模泛化，模型在训练未见的大规模实例上仍保持性能优势，突破传统启发式方法的局限性。

3.1.3 LEARNING CUT SELECTION FOR MIXED-INTEGER LINEAR PROGRAMMING VIA HIERARCHICAL SEQUENCE MODEL

作者： Zhihai Wang, Xijun Li, Jie Wang, Yufei Kuang, Mingxuan Yuan, Jia Zeng, Yongdong Zhang, Feng Wu

方法详解： 该方法[WLW+23]针对割平面法在组合优化问题中的应用提出了三个问题：

- (P1) 需要选择哪些平面？
- (P2) 需要选择多少个平面？
- (P3) 割平面的顺序应该是什么样的？

该论文提出了一种分层顺序模型（hierarchical sequence model, HEM），利用分层强化学习方法来解决上述问题：上层模型决定切个平面的数量；下层模型决定切割平面的顺序。针对上述三个问题，这里使用两个不同的方法进行验证，分别是RandomAll和RandomNV。RandomAll随机排列所有候选切割，并按随机顺序将所有切割添加到LP松弛中。RandomNV在RandomAll的基础上，增加了一个选择割平面比例的因子，并根据Normalized Violation Score进行选择。

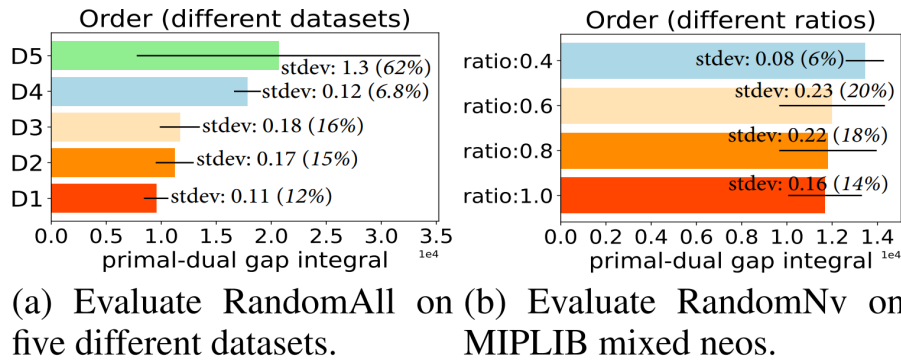


Figure 1: RandomAll/RandomNV问题验证

观察结果可以发现，在不同数据集上，(a)图中，计算结果的方差较大；(b)图中，计算结果的方差较大，且平面选择的比例会对求解精度造成影响。

因此，有必要决定割平面选择的比例以及进行割平面的顺序。

分层强化学习方法设计-MDP

- **state:** 包含三部分内容，当前LP松弛问题的数学模型、候选平面集合、LP松弛问题最优解
- **action:** 所有候选平面顺序排列的子集
- **reward:** (i) 在线性规划松弛求解结束时收集的指标，例如对偶界的提升；(ii) 或在整个求解过程结束时的统计数据，例如求解时间和原始-对偶间隙积分
- **transition function:** 转移函数将当前状态 s 和动作 a 映射到下一个状态 s' ，其中 s' 表示通过在当前线性规划松弛中加入所选割平面后生成的下一个线性规划松弛

分层强化学习方法设计-上层/下层模型

上层策略 $\pi_h(\cdot|s)$: 通过预测一个合适的比例 p ，来学习应选择多少个割平面。

下层策略 $\pi_l(\cdot|s)$: 学习在高层策略确定的数量基础上，选择一个有序的割平面子集。

最后，根据全概率公式推导出割平面选择策略，即 $\pi(a_k|s) = E_{k \sim \pi_h(\cdot|s)}[\pi_l(a_k|s; k)]$

其中， k 表示给定的比例， a_k 表示对应的动作。

分层强化学习方法设计-训练

对于割平面选择任务，我们的目标是找到参数 θ ，使在所有轨迹上的期望回报最大化。

$$J(\theta) = E_{s \sim \mu, a_k \sim \pi_\theta(\cdot|s)}[r(s, a_k)]$$

$$\nabla_{\theta_1} J([\theta_1, \theta_2]) = E_{s \sim \mu, k \sim \pi_{\theta_1}^h(\cdot|s)} \left[\nabla_{\theta_1} \log(\pi_{\theta_1}^h(k|s)) E_{a_k \sim \pi_{\theta_2}^l(\cdot|s, k)}[r(s, a_k)] \right],$$

$$\nabla_{\theta_2} J([\theta_1, \theta_2]) = E_{s \sim \mu, k \sim \pi_{\theta_1}^h(\cdot|s), a_k \sim \pi_{\theta_2}^l(\cdot|s, k)} \left[\nabla_{\theta_2} \log \pi_{\theta_2}^l(a_k|s, k) r(s, a_k) \right].$$

创新点总结:

- 从大量的实证结果中观察到，选择切割的顺序对解决MILP的效率有显著影响
- 提出的HEM是第一个能够从数据驱动的角度同时处理(P1)-(P3)的方法。
- 提出将切段选择任务表述为一个序列到序列的学习问题，不仅可以捕获底层的顺序信息，还可以很好地捕获切段之间的相互作用，以选择相互补充的切段

3.2 相关代码介绍

3.2.1 Hybrid Models for Learning to Branch (NeurIPS 2020)

代码仓库: <https://github.com/pg2455/Hybrid-learn2branch> 该项目聚焦于混合整数线性规划(MILP)问题，通过学习分支定界算法中的变量选择策略，以提升求解MILP问题的效率。项目使用了PyTorch等工具，并且提供了多种模型的训练和评估脚本。代码结构如下:

- **models/:** 包含不同模型（如GCNN、MLP、混合模型等）的定义和相关代码。
- **03_train_gcnn_torch.py:** 用于训练PyTorch版本的GCNN模型，可指定问题类型、模型名称、随机种子等参数。
- **03_train_hybrid.py:** 负责训练混合模型，支持不同的模型架构（如concat、film），可选择是否使用预训练模型、知识蒸馏等。
- **04_test_gcnn_torch.py:** 用于测试GCNN模型的性能，可指定问题类型、GPU使用情况等参数。

- `05_evaluate_gcnn_torch.py`: 评估GCNN 模型在不同MILP 实例上的性能，可指定问题类型、GPU 使用情况、模型名称等参数。
- `utilities_gcnn_torch.py`: 提供GCNN 数据集的定义和批量加载函数。
- `README.md`: 包含项目的安装说明、运行指令等信息。
- `RESULTS.md`: 提供了复现评估结果的相关指令。

References

- [BDZ21] Amin Banitalebi-Dehkordi and Yong Zhang. Ml4co: is gcnn all you need? graph convolutional neural networks produce strong baselines for combinatorial optimization problems, if tuned and trained properly, on appropriate data. *arXiv preprint arXiv:2112.12251*, 2021.
- [GCC⁺22] Maxime Gasse, Quentin Cappart, Jonas Charfreitag, Laurent Charlin, Didier Ch’etelat, Antonia Chmiela, Justin Dumouchelle, Ambros M. Gleixner, Aleksandr M. Kazachkov, Elias Boutros Khalil, Pawel Lichocki, Andrea Lodi, Miles Lubin, Chris J. Maddison, Christopher Morris, Dimitri J. Papageorgiou, Augustin Parjadis, Sebastian Pokutta, Antoine Prouvost, Lara Scavuzzo, Giulia Zarpellon, Linxin Yangm, Sha Lai, Akang Wang, Xiaodong Luo, Xiang Zhou, Haohan Huang, Sheng Cheng Shao, Yuanming Zhu, Dong Zhang, Tao Manh Quan, Zixuan Cao, Yang Xu, Zhewei Huang, Shuchang Zhou, Cheng Binbin, He Minggui, Hao Hao, Zhang Zhiyu, An Zhiwu, and Mao Kun. The machine learning for combinatorial optimization competition (ml4co): Results and insights. *ArXiv*, abs/2203.02433, 2022.
- [GCF⁺19] Maxime Gasse, Didier Ch’etelat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, 32, 2019.
- [GGK⁺20] Prateek Gupta, Maxime Gasse, Elias Khalil, Pawan Mudigonda, Andrea Lodi, and Yoshua Bengio. Hybrid models for learning to branch. *Advances in neural information processing systems*, 33:18087–18097, 2020.
- [KLBS⁺16] Elias Khalil, Pierre Le Bodic, Le Song, George Nemhauser, and Bistra Dilkina. Learning to branch in mixed integer programming. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- [LCK18] Zhuwen Li, Qifeng Chen, and Vladlen Koltun. Combinatorial optimization with graph convolutional networks and guided tree search. *Advances in neural information processing systems*, 31, 2018.
- [NOST18] Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takác. Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems*, 31, 2018.
- [WLW⁺23] Zhihai Wang, Xijun Li, Jie Wang, Yufei Kuang, Mingxuan Yuan, Jia Zeng, Yongdong Zhang, and Feng Wu. Learning cut selection for mixed-integer linear programming via hierarchical sequence model. *arXiv preprint arXiv:2302.00244*, 2023.